

LLM Server

仕様書

User-Centric Spatial Referring for Tabletop Robot Manipulation

with MR Grounding and LLMs

バージョン: 1.0

作成日: 2026-02-28

目次

目次	2
1. システム概要	4
1.1 役割と責務	4
1.2 技術スタック	4
2. 通信プロトコル	5
2.1 WebSocket エンドポイント	5
2.2 メッセージフォーマット	5
2.2.1 クライアント → サーバー: 空間参照リクエスト	5
2.2.2 サーバー → クライアント: 推論結果	6
2.2.3 クライアント → サーバー: 修正リクエスト	6
2.2.4 クライアント → サーバー: 確認完了	6
2.2.5 サーバー → クライアント: ロボット動作指令	6
3. 空間特徴量の算出	7
3.1 座標系の定義	7
3.2 算出する特徴量	7
3.2.1 共通特徴量（フレーム非依存）	7
3.2.2 フレーム依存特徴量（ユーザー視点）	7
3.2.3 フレーム依存特徴量（ロボット視点）	8
3.2.4 実行フェーズ専用データ	8
3.3 算出アルゴリズム	9
4. LLM プロンプト設計	10
4.1 2 ステージアーキテクチャ	10
4.2 Stage 1: Interpreter Prompt	10
4.3 Stage 2: ObjectSelector Prompt	10
4.4 Spatial Context フォーマット	11
4.5 リファインメント時の追加コンテキスト	11
5. 処理パイプライン	12
5.1 初回リクエスト処理フロー	12
5.2 リファインメント処理フロー	12
5.3 確認・実行フロー	12
6. エラーハンドリング	13
6.1 フォールバック戦略	13
7. 設定パラメータ	14
7.1 設定ファイル例 (config.yaml)	14

8. 性能要件	15
---------------	----

1. システム概要

本仕様書は、MR 空間参照表現を用いたテーブルトップロボット操作システムにおける LLM サーバーの設計と実装要件を定義する。LLM サーバーは、Unity (HoloLens 2) クライアントからの空間情報と音声テキストを受け取り、LLM 推論によりターゲット物体を特定し、結果をクライアントに返すバックエンドシステムである。

1.1 役割と責務

- Unity クライアントとの WebSocket 通信の管理
- Interpreter: 発話テキストから参照フレーム (user/robot) の判定
- 選択されたフレームに基づく空間特徴量の算出
- ObjectSelector: 発話と空間特徴量に基づく候補物体のランク付け
- 修正発話に基づく再推論 (リファインメントループ)

1.2 技術スタック

項目	技術
言語	Python 3.10+
通信	WebSocket (websockets ライブラリ)
LLM	OpenAI GPT-4 API / ローカル LLM (LLaMA 等)
数値計算	NumPy
座標変換	SciPy (空間変換)
設定管理	YAML / JSON

2. 通信プロトコル

2.1 WebSocket エンドポイント

エンドポイント	方向	説明
ws://<host>:8765/spatial	Client → Server	空間情報とユーザー発話の送信
ws://<host>:8765/result	Server → Client	推論結果の返送
ws://<host>:8765/status	双方向	接続状態・エラー通知

2.2 メッセージフォーマット

すべてのメッセージは JSON 形式で送受信される。

2.2.1 クライアント → サーバー: 空間参照リクエスト

```
{
  "type": "spatial_reference_request",
  "request_id": "uuid-v4",
  "timestamp": "ISO8601",
  "utterance": {
    "text": "右側の赤い箱を取って",
    "language": "ja"
  },
  "user_pose": {
    "position": { "x": 0.5, "y": 1.2, "z": -0.8 },
    "forward": { "x": 0.0, "y": 0.0, "z": 1.0 },
    "up": { "x": 0.0, "y": 1.0, "z": 0.0 }
  },
  "objects": [
    {
      "id": "obj_001",
      "label": "box",
      "color": "red",
      "position": { "x": 0.3, "y": 0.75, "z": 0.2 },
      "bounding_box": {
        "min": { "x": 0.2, "y": 0.7, "z": 0.1 },
        "max": { "x": 0.4, "y": 0.8, "z": 0.3 }
      }
    }
  ],
  "table_frame": {
    "origin": { "x": 0, "y": 0.7, "z": 0 },
    "x_axis": { "x": 1, "y": 0, "z": 0 },
    "z_axis": { "x": 0, "y": 0, "z": 1 }
  },
  "robot_pose": {
    "position": { "x": -0.5, "y": 0.7, "z": 0 },
    "forward": { "x": 1, "y": 0, "z": 0 }
  }
}
```

2.2.2 サーバー → クライアント: 推論結果

```
{
  "type": "spatial_reference_result",
  "request_id": "uuid-v4",
  "target": {
    "object_id": "obj_001",
    "confidence": 0.92,
    "reference_frame": "user_egocentric"
  },
  "ranked_candidates": [
    { "object_id": "obj_001", "score": 0.92 },
    { "object_id": "obj_003", "score": 0.55 }
  ],
  "reasoning": "ユーザー視点で右側かつ赤色の物体"
}
```

2.2.3 クライアント → サーバー: 修正リクエスト

```
{
  "type": "refinement_request",
  "request_id": "uuid-v4",
  "original_request_id": "uuid-v4-original",
  "utterance": {
    "text": "もっと奥のやつ",
    "language": "ja"
  },
  "user_pose": { ... },
  "previous_target": "obj_001"
}
```

2.2.4 クライアント → サーバー: 確認完了

```
{
  "type": "confirmation",
  "request_id": "uuid-v4",
  "confirmed_object_id": "obj_001",
  "action": "pick"
}
```

2.2.5 サーバー → クライアント: ロボット動作指令

```
{
  "type": "robot_command",
  "request_id": "uuid-v4",
  "action": "pick",
  "target_object_id": "obj_001",
  "target_position": { "x": 0.3, "y": 0.75, "z": 0.2 },
  "status": "executing"
}
```

3. 空間特徴量の算出

本システムの LLM 推論は 2 段階で構成される。第 1 段階の **Interpreter** が発話から参照フレーム（`user_egocentric / robot_centric`）を判定し、第 2 段階の **ObjectSelector** は選択されたフレームの特徴量のみを用いて候補物体をランク付けする。したがって、**ObjectSelector** には **Interpreter** が選択したフレームに対応する空間特徴量のみを渡す。両フレームの特徴量を同時に渡すと、選択されなかったフレームの情報がノイズとなり推論精度を低下させるためである。

3.1 座標系の定義

座標系	原点	説明
テーブル座標系	AR マーカー位置	統一座標系。全てのエンティティはこの座標系に変換される
ユーザー相対座標系	ユーザーの頭部位置	ユーザーの視線方向を基準とした前後左右の計算に使用
ロボット相対座標系	ロボットベース位置	確認後の把持実行フェーズでのみ使用

3.2 算出する特徴量

設計方針: **Interpreter** が選択した参照フレームに応じて、対応するフレーム依存特徴量のみを算出し **ObjectSelector** に渡す。共通特徴量はフレーム選択に依存せず常に付与する。

3.2.1 共通特徴量（フレーム非依存）

参照フレームの選択結果にかかわらず、常に各候補物体に付与される属性情報。

特徴量	型	説明
label	string	物体のカテゴリラベル (例: box, cup, bottle)
color	string	物体の色 (例: red, blue)
cluster_id	int	空間クラスタリングによるグループ ID

3.2.2 フレーム依存特徴量（ユーザー視点）

Interpreter が `user_egocentric` フレームを選択した場合に算出される。ユーザーの頭部位置と視線方向を基準とする。

特徴量	型	単位	説明
relative_x	float	m	ユーザー基準の左右座標 (右: +, 左: -)
relative_y	float	m	ユーザー基準の上下座標 (上: +)
relative_z	float	m	ユーザー基準の前後座標 (前方: +)
distance	float	m	ユーザーから物体までのユークリッド距離
azimuth_angle	float	deg	ユーザー正面方向を 0°とした水平角 (左: -, 右: +)
depth_rank	int	-	ユーザーから見た前後方向の順位 (1=最手前)
lateral_rank	int	-	ユーザーから見た左右方向の順位 (1=最左)
is_closest	bool	-	最近傍物体フラグ
relative_position	string	-	定性的位置記述 (例: front-right)

3.2.3 フレーム依存特徴量（ロボット視点）

Interpreter が robot-centric フレームを選択した場合に算出される。ロボットベースの位置と正面方向を基準とする。

特徴量	型	単位	説明
relative_x	float	m	ロボット基準の左右座標 (右: +, 左: -)
relative_y	float	m	ロボット基準の上下座標 (上: +)
relative_z	float	m	ロボット基準の前後座標 (前方: +)
distance	float	m	ロボットから物体までのユークリッド距離
azimuth_angle	float	deg	ロボット正面方向を 0°とした水平角 (左: -, 右: +)
depth_rank	int	-	ロボットから見た前後方向の順位 (1=最手前)
lateral_rank	int	-	ロボットから見た左右方向の順位 (1=最左)
is_closest	bool	-	最近傍物体フラグ
relative_position	string	-	定性的位置記述 (例: front-left)

3.2.4 実行フェーズ専用データ

以下のデータは空間参照の解決（Interpreter / ObjectSelector）には使用しない。ユーザーが確認を完了しロボットが把持動作を実行する段階でのみ参照される。

データ	型	単位	説明
robot_distance	float	m	ロボットから確認済みターゲットまでの距離

grasp_position	Vector3	m	ロボット座標系での把持位置
reachability	bool	-	ロボットの到達可能範囲内フラグ

3.3 算出アルゴリズム

空間特徴量の算出は、Interpreter の出力（参照フレーム判定）に基づき以下の手順で行う：

Step 1: 参照フレーム判定 (Interpreter) — LLM が発話テキストを解析し、空間参照フレーム（user_egocentric / robot_centric）を決定する。

Step 2: 座標変換 — 選択されたフレームの基準点（ユーザー頭部 or ロボットベース）に基づき、テーブル座標系上の全物体位置を相対座標系に変換する。

Step 3: 距離・角度計算 — 選択されたフレームの相対座標系において、各物体の距離、方位角を算出する。

Step 4: ランク付け — 奥行き方向・左右方向それぞれの軸に沿って物体を順序付ける。

Step 5: 定性記述生成 — 方位角と奥行きランクから定性的な位置記述（front-left, back-right 等）を生成する。

Step 6: ObjectSelector へ渡す — 共通特徴量 + 選択フレームの特徴量のみを構造化データとして ObjectSelector に渡す。

4. LLM プロンプト設計

本システムの LLM 推論は 2 つのステージで構成される。各ステージは別々のプロンプトで実行され、それぞれ異なる役割を担う。

4.1 2 ステージアーキテクチャ

ステージ	名称	入力	出力
Stage 1	Interpreter	発話テキスト	参照フレーム (user_egocentric / robot_centric)
Stage 2	ObjectSelector	発話 + 選択フレームの空間特徴量	ランク付きターゲット候補

4.2 Stage 1: Interpreter Prompt

発話から参照フレームを判定する。物体情報は渡さない。

あなたは空間参照フレーム判定エージェントです。
ユーザーの発話を分析し、どの視点を基準とした空間参照表現かを判定してください。

判定基準:

- 「(私の)右」「手前」「目の前」等 → user_egocentric
- 「ロボットの右」「ロボットに近い」等 → robot_centric
- 色や名前のみ (空間表現なし) → user_egocentric (デフォルト)

出力形式 (JSON):

```
{
  "reference_frame": "user_egocentric | robot_centric",
  "confidence": 0.0-1.0,
  "spatial_keywords": ["右", "手前"]
}
```

4.3 Stage 2: ObjectSelector Prompt

Interpreter の結果に基づき、選択されたフレームの空間特徴量のみを用いてターゲットを特定する。

あなたはテーブルトップ環境における物体選択エージェントです。
参照フレームは既に確定しています: {reference_frame}
以下の候補物体から、ユーザーの発話に最も合致する物体を選択してください。

出力形式 (JSON):

```
{
  "ranked_objects": [
    { "object_id": "...", "score": 0.0-1.0, "reason": "..." }
  ]
}
```

```
}
```

4.4 Spatial Context フォーマット

ObjectSelector に渡す物体情報は、Interpreter が選択したフレームの特徴量のみを含む。以下は user_egocentric フレーム選択時の例:

```
=== 参照フレーム: user_egocentric ===
=== 候補物体一覧 ===
物体 ID: obj_001
  ラベル: box, 色: red
  相対座標: (x: +0.25, y: +0.05, z: +0.38)
  ユーザーからの距離: 0.45m
  方位角: +35° (ユーザーの右側)
  奥行きランク: 2/5 (手前から 2 番目)
  左右ランク: 4/5 (左から 4 番目)
  定性位置: front-right
---
物体 ID: obj_002
  ラベル: box, 色: blue
...
```

4.5 リファインメント時の追加コンテキスト

ユーザーが修正発話を行った場合、プロンプトに以下の情報を追加する:

```
=== 前回の推論結果 ===
前回のターゲット: obj_001 (box, red, front-right)
修正発話: 「もっと奥のやつ」
→ 前回のターゲットより奥にある候補を優先してスコアリングしてください。
```

5. 処理パイプライン

5.1 初回リクエスト処理フロー

1. WebSocket メッセージ受信 (spatial_reference_request)
2. 入力バリデーション（必須フィールドの確認、座標値の範囲チェック）
3. [Stage 1: Interpreter] 発話テキストを LLM に送信し、参照フレームを判定
4. 判定された参照フレームに基づき、対応する基準点（ユーザー頭部 or ロボットベース）で座標変換
5. 選択フレームの空間特徴量のみを算出（共通特徴量 + フレーム依存特徴量）
6. [Stage 2: ObjectSelector] 発話テキスト + 空間特徴量を LLM に送信し、ターゲットをランク付け
7. LLM レスポンスのパーズとバリデーション
8. 推論結果をクライアントに返送 (spatial_reference_result)

5.2 リファインメント処理フロー

1. 修正リクエスト受信 (refinement_request)
2. 前回のコンテキスト（ターゲット ID、元の発話）をセッションから復元
3. 修正発話を含む拡張プロンプトを構築
4. LLM 再推論
5. 更新された結果をクライアントに返送

5.3 確認・実行フロー

1. 確認メッセージ受信 (confirmation)
2. 確認された物体 ID の検証
3. ロボット座標系での把持位置を算出
4. ロボットコントローラーに動作指令を発行
5. 動作状態をクライアントに通知 (robot_command)

6. エラーハンドリング

エラーコード	状況	対応
E001	LLM API タイムアウト	リトライ（最大 3 回）後、エラー応答を返す
E002	LLM レスポンスのパース失敗	デフォルトの距離ベースフォールバックを使用
E003	物体リストが空	クライアントに物体未検出エラーを返す
E004	無効な座標値	入力バリデーションエラーを返す
E005	WebSocket 切断	セッション情報を保持し再接続を待機
E006	セッション期限切れ	クライアントに再初期化を要求

6.1 フォールバック戦略

LLM が利用できない場合、以下のルールベースのフォールバックロジックを使用する：

- 色情報が発話に含まれる場合: 色の一致でフィルタリング
- 「右」「左」等の方向語がある場合: 方位角ベースでソート
- 「手前」「奥」等の距離語がある場合: 奥行きランクベースでソート
- 「一番近い」等の最上級表現: 距離の昇順でソート

7. 設定パラメータ

パラメータ	型	デフォルト値	説明
ws_host	string	0.0.0.0	WebSocket バインドアドレス
ws_port	int	8765	WebSocket ポート
llm_model	string	gpt-4	使用する LLM モデル名
llm_timeout_sec	float	10.0	LLM API タイムアウト (秒)
llm_max_retries	int	3	LLM API リトライ回数
llm_temperature	float	0.1	LLM サンプリング温度
session_timeout_sec	float	300.0	セッション有効期限 (秒)
max_candidates	int	20	LLM に渡す候補物体の上限数
azimuth_bin_deg	float	15.0	方位角の離散化ビン幅
depth_bin_m	float	0.05	奥行きの離散化ビン幅

7.1 設定ファイル例 (config.yaml)

```

server:
  host: 0.0.0.0
  port: 8765

llm:
  model: gpt-4
  timeout: 10.0
  max_retries: 3
  temperature: 0.1

spatial:
  max_candidates: 20
  azimuth_bin: 15.0
  depth_bin: 0.05

session:
  timeout: 300

```

8. 性能要件

指標	目標値	備考
空間特徴量算出時間	< 50 ms	20 物体まで
LLM 推論時間 (E2E)	< 3 秒	ネットワーク遅延含む
WebSocket 往復遅延	< 100 ms	ローカルネットワーク
同時セッション数	≥ 5	マルチユーザー対応
リファインメント応答	< 2 秒	コンテキスト再利用により高速化